



SDI Investment Property Marketplace

System Documentation

What has been built, how to run it, and who can see what

Repository	github.com/neilgreene/property-management
Branch	main
Commits	13
Database	PostgreSQL 16 (row-level security; requires 15+)
Application	Node.js 18+ (no framework, no runtime dependencies beyond pg)
Automated checks	63 worker tests plus 4 SQL walkthroughs, all passing
Deployment status	Local development only. Not deployed, not internet-facing.

Every fact in this document was extracted from the repository and from a freshly built database, not written from memory. Regenerate it with `python3 docs/generate_system_documentation.py`.

This page intentionally left blank.

Contents

1. What This Is	4
1.1 The problem it solves first	4
1.2 Three visibility bands	4
2. How To Run It	4
2.1 Docker — nothing installed but Docker	4
2.2 Local PostgreSQL 16+ and Node 18+	4
2.3 Individual pieces	5
2.4 The integration worker	5
3. Users and Access	6
3.1 There is no login system yet	6
3.2 Seeded people	6
3.3 Database roles	6
4. Architecture	8
4.1 Four schemas, and the separation is load-bearing	8
4.2 Tables	8
5. The API	8
5.1 Database views — what the application reads	8
5.2 Callable functions	9
5.3 HTTP endpoints	9
6. The GoHighLevel Integration	10
6.1 What the worker does	10
6.2 The fee gate has two conditions, not one	10
6.3 Direction decides what happens to an inbound event	10
6.4 One item is unverified	10
7. What Is Tested	10
8. What Is Not Built	11
9. Where Things Are	11

1. What This Is

A working system for a private investment property marketplace: vetted properties listed with financial analysis, browsable without revealing addresses, with the full detail unlocked only after an investor signs and pays the platform fee agreement. Agents see only their own assignments. Staff see everything.

1.1 The problem it solves first

The commercial model depends on one thing holding: an unpaid visitor must not be able to obtain a property's street address. Everything else is ordinary software; that is not. So the visibility rules are enforced **inside the database**, by row-level security policies and column grants, rather than by application code.

This matters because it changes what an attacker has to defeat. If the rule lives in application code, anyone who finds an unguarded query, an API bug, or a direct database connection gets the address. Here, the address is withheld by the database itself: a caller who writes their own SQL, bypasses the application entirely, and connects directly still cannot read it.

1.2 Three visibility bands

Band	Contains	Released to	Enforced by
1 — Public	City, price, cap rate, NOI, beds, baths, year built	Anyone, including anonymous visitors	RLS policy per role
2 — Gated	Street address, unit, parcel number, seller disclosure, true coordinates	Investors who signed AND paid; the assigned agent; staff	Masking in a <code>security_invoker</code> view, keyed on a data predicate
3 — Internal	Acquisition cost, source channel, staff notes, margin	Staff only	Column-level GRANT (a hard ACL)

Coordinates degrade rather than disappear. An ungated viewer gets a deterministic offset of roughly one kilometre, seeded on the property id, so a map still renders a neighbourhood and repeated page loads cannot be averaged to recover the true point.

2. How To Run It

2.1 Docker — nothing installed but Docker

```
git clone https://github.com/neilgreene/property-management
cd property-management
docker compose up
```

Builds the database from `sql/`, seeds it, and serves the demo at **`http://localhost:3000`**. PostgreSQL is exposed on `localhost:5432` (database `sdi`, user `postgres`, password `postgres`). `docker compose down -v` discards the database.

2.2 Local PostgreSQL 16+ and Node 18+

```
./run.sh
```

Loads all eleven schema files, runs all four SQL walkthroughs, runs the 63 worker tests, then starts the demo. It assumes `psql` and `createdb` work as your own user, which is the default for PostgreSQL.app and Homebrew.

2.3 Individual pieces

Command	What it does
<code>psql -d sdi -f sql/05_tests.sql</code>	Security walkthrough: 11 checks, 5 of them attacks
<code>psql -d sdi -f sql/07_ghl_tests.sql</code>	GoHighLevel bridge: 7 checks
<code>psql -d sdi -f sql/10_review_tests.sql</code>	Review queue actions: 7 checks
<code>psql -d sdi -f sql/14_pipeline_tests.sql</code>	Deal visibility and stage history: 9 checks
<code>cd worker && npm test</code>	63 unit and end-to-end tests

2.4 The integration worker

Deliberately not started by a bare `docker compose up`, because it needs real GoHighLevel credentials and there is no point running it without them.

```
GHL_TOKEN=... GHL_LOCATION_ID=... docker compose --profile worker up
```

Variable	Where it comes from
<code>GHL_TOKEN</code>	GoHighLevel: Settings → Private Integrations. Scoped to an entire sub-account.
<code>GHL_LOCATION_ID</code>	The sub-account id, visible in the GoHighLevel URL
<code>GHL_WEBHOOK_PUBLIC_KEY</code>	GoHighLevel's published webhook verification key (PEM)

The token is read from the environment and is never written into any file in the repository. It grants access to the whole sub-account — contacts, invoices, transactions — so it must never reach browser-side code.

3. Users and Access

3.1 There is no login system yet

This must be stated plainly, because it is the most likely thing to be misread. The system has no authentication: no password for a person, no session, no sign-in page. The demo presents a persona switcher, and choosing a persona makes the web tier assume the matching database role and set that person's id for the duration of one transaction.

That is deliberate for this stage and costs nothing later. The database contract is identical either way: in production the persona and actor id come out of an authenticated session instead of a dropdown, and not one policy, view or grant changes. Authentication is the missing piece — the authorisation model beneath it is complete and tested.

3.2 Seeded people

Created by `sql/04_seed.sql`. These are demonstration records, not real people; the addresses and financials attached to them are invented.

Name	Role	Email	Fee agreement	Brand	What they demonstrate
Jessica Pool	admin	jpool@yahoo.com	n/a	BRAND_A	Full staff access, all bands
Dan Beitor	admin	dan@example.com	n/a	BRAND_A	Full staff access, all bands
Tom Bradbury	agent	tom@example.com	n/a	BRAND_A	4 assigned properties, incl. an unpublished draft
Priya Raman	agent	priya@example.com	n/a	BRAND_A	A second agent, to prove agents are isolated
Ruth Okonkwo	investor	ruth@example.com	signed	BRAND_A	Gate open: sees addresses and true coordinates
Marcus Pell	investor	marcus@example.com	not signed	BRAND_A	Gate shut: same query, address withheld
Ines Duarte	investor	ines@example.com	signed	KAVADOO	The second brand, at concierge pricing

Ruth and Marcus are the pair that carries the argument: identical role, identical SQL, and the only difference between them is one timestamp in a data column. The address appears for one and not the other with no application logic involved at all.

3.3 Database roles

Application roles are created NOLOGIN and passwordless on purpose. `sdi_app` is NOINHERIT, so it holds no privileges of its own and must assume exactly one persona role per transaction — the application cannot forget to assume a role and accidentally run with more authority than it should.

Role	Purpose	Login	Demo password
<code>sdi_app</code>	The only role the web tier connects as	yes	<code>demo_app_pw</code>
<code>sdi_public</code>	Anonymous visitors. Band 1 only	no	assumed via SET ROLE

Role	Purpose	Login	Demo password
sdi_investor	Investors. Band 2 if the gate is open	no	assumed via SET ROLE
sdi_agent	Agents. Band 2 on assigned properties	no	assumed via SET ROLE
sdi_admin	Staff. All three bands	no	assumed via SET ROLE
sdi_integration	The GoHighLevel worker. No access to core at all	yes	demo_int_pw
sdi_test_admin	Test fixtures only. BYPASSRLS	yes	demo_test_pw

Those passwords come from `sql/99_local_logins.sql`, exist so the demo stack is connectable, and are published in a public repository. They are worth exactly nothing and that file must not be loaded anywhere that matters. `sdi_test_admin` carries BYPASSRLS and belongs only in a test database.

4. Architecture

```

browser ■■■ web/server.js ■■■ api.* ■■■ core.* (PostgreSQL)
          sdi_app          views      tables + RLS
          SET ROLE          ■
                          ■■■■ sec.* predicates

GoHighLevel ■■■webhook■■■ worker/src/index.js ■■■ ghl.* (separate schema,
          ■■■REST■■■■ sdi_integration          no access to core)

```

4.1 Four schemas, and the separation is load-bearing

Schema	Holds	Who has USAGE
core	Base tables. Properties, people, deals, assignments	No application role. Name resolution fails before any ACL or policy is consulted, so there is no path around the views.
sec	Security predicates, definer-rights	All persona roles. They live here because a security_invoker view resolves the functions it calls as the caller too, not just the tables.
api	Masking views and the write path	All persona roles. The only surface the application reads.
ghl	The GoHighLevel bridge	sdi_integration only. The web tier cannot enumerate CRM contacts, invoices or transactions.

Brand is a lens, not a property attribute. Both brands read the same core.property rows; core.property_brand decides publication and price per brand. Adding the second brand is rows in one table, not a schema refactor.

4.2 Tables

Schema	Tables
core	brand, person, property, property_brand, property_assignment, saved_property, pipeline, pipeline_stage, deal, deal_stage_history
ghl	id_map, webhook_event, transaction, fee_agreement, sync_state, outbox, review_queue, reviewable_field

All ten core tables have row-level security enabled and forced.

5. The API

5.1 Database views — what the application reads

View	Returns	Granted to
api.property	Properties with band 2 masked or released per caller	public, investor, agent, admin
api.property_internal	Band 3: acquisition cost, source, notes, margin	admin only
api.my_saved	The caller's own saved properties	investor, admin
api.deal	Deals the caller is party to, with stage and age	investor, agent, admin

View	Returns	Granted to
<code>api.deal_history</code>	Stage transitions for those deals	investor, agent, admin
<code>api.review_open</code>	Inbound CRM edits awaiting a decision	admin only

Note that `api.deal` is not granted to `sdi_public` at all. A deal is never public, at any stage.

5.2 Callable functions

Function	Does	Caller
<code>api.save_property(uuid)</code>	Saves a property to the caller's list, rejecting anything they cannot see	investor
<code>api.review_decide(bigint, text)</code>	Accept or reject a queued CRM edit; accepting writes an allowlist of columns only	admin
<code>api.security_invariants()</code>	Returns zero rows, or names what is broken	admin
<code>ghl.apply_fee_agreement(text)</code>	Opens the gate for a person, if the document is completed AND paid	integration worker

The `sec.*` predicates (`can_see_address`, `can_see_deal`, `is_assigned`, `actor`, `jitter` and others) are internal. They are the shared source of truth that policies, views and write functions all consult, so a rule cannot drift between the place it is read and the place it is enforced.

5.3 HTTP endpoints

Service	Method and path	Purpose
Web demo	GET /	The demo interface
Web demo	GET <code>/api/view?persona=&brand=</code>	Everything one persona sees, in one payload
Web demo	GET <code>/api/probe?persona=</code>	Attempts a direct base-table read, to show the refusal
Worker	POST <code>/webhooks/ghl</code>	Receives a GoHighLevel delivery, verifies its signature
Worker	GET <code>/healthz</code>	Queue depth: pending events, bad signatures, outbox backlog, open reviews

The web demo runs on port 3000, the worker on 3001. Neither is authenticated; neither should be exposed to a network until section 8 is addressed.

6. The GoHighLevel Integration

Built against GoHighLevel's official OpenAPI specifications. The full API contract is documented separately in `docs/GHL-Interface-Specification.pdf` (17 pages).

6.1 What the worker does

Component	Responsibility
<code>ghlClient.js</code>	HTTP client. Sets the required <code>Version: 2021-07-28</code> header once, paces under the 100-request/10-second limit, retries 429 and 5xx, never retries 403
<code>signature.js</code>	Verifies webhook signatures against GoHighLevel's published RSA public key
<code>webhookReceiver.js</code>	Records deliveries, deduplicating on <code>webhookId</code> and rejecting stale ones
<code>handlers.js</code>	Dispatches received events
<code>outbox.js</code>	Drains outbound writes with retry that cannot duplicate a record
<code>sync/documents.js</code>	Polls fee agreement state
<code>sync/transactions.js</code>	Nightly reconciliation of payments
<code>migrate/</code>	EspoCRM to GoHighLevel load, in ordered passes

6.2 The fee gate has two conditions, not one

GoHighLevel reports document status and payment status *independently*. A document can be signed and unpaid. `ghl.apply_fee_agreement()` is the only thing in the system that opens the gate, and it requires both: completed or accepted, **and** paid. A tag-based unlock gets this wrong.

6.3 Direction decides what happens to an inbound event

`core.property` is the system of record; GoHighLevel is downstream of it for listings. So an inbound record edit means somebody changed a property inside the CRM, against the grain of the architecture. Applying it would let the CRM silently overwrite the authoritative row. Dropping it would lose a real edit. It is neither: it goes to `ghl.review_queue` for a person to decide.

Events that genuinely originate in GoHighLevel — a signed document, a settled payment, a contact created by staff — are applied directly, because for those GoHighLevel is the source. Accepting a queued edit writes only an allowlist of band 1 columns, so a status change can never become a route to rewriting an address or a cost basis.

6.4 One item is unverified

GoHighLevel publishes the webhook public key but does not document the signature algorithm. The code uses PKCS#1 v1.5 with SHA-256, the conventional pairing for that key format. One captured live delivery would settle it, and until then the receiver should not be trusted in production. `worker/tools/capture-webhook.js` exists to capture one: run it somewhere GoHighLevel can reach, send a test contact through, and it writes the raw bytes and headers untouched.

7. What Is Tested

Not a coverage percentage. These are the specific claims that are checked, and the attacks that are run against them.

Suite	Checks	Notable
sql/05_tests.sql	11	Five attacks: direct base-table read, another investor's saved list, saving an invisible property, a VOLATILE predicate side-channel, and the standing invariants
sql/07_ghl_tests.sql	7	A signed-but-unpaid document must not open the gate; replay must not move a signature timestamp
sql/10_review_tests.sql	7	A payload smuggling an address and a cost basis alongside a legitimate status change; only the allowlist is applied
sql/14_pipeline_tests.sql	9	One agent cannot read another's deal by naming its id; one investor cannot read another's stage history
worker/ (npm test)	63	Signature forgery, replay, oversized bodies, rate-limit handling, ambiguous-create duplication, migration resumability

`api.security_invariants()` must always return zero rows. It catches the four changes that quietly dismantle the model: USAGE granted on core, an internal column exposed to a non-admin, RLS switched off on a protected table, or a view created without `security_invoker`. Wire it into CI and a nightly check.

8. What Is Not Built

Stated plainly so nothing here is mistaken for finished.

Missing	Consequence
Authentication	No login, no sessions, no passwords for people. The demo selects a persona. This is the gap between the current system and anything user-facing.
The public marketplace UI	Only the demo interface exists. It exercises the model; it is not the product.
Document storage	The signed PDF artifact is not stored
Messaging and unified inbox	Not started
Audit trail on band 2 and 3 reads	Who viewed which address, and when, is not recorded
Co-investment matching	The pipeline exists; the matching engine does not
The external status scraper	Not built. The review queue that would receive its output is.
EspoCRM field mapping	The load ordering and resumability are built and tested. The field-level mapping needs the live EspoCRM schema.
Any deployment	Nothing is hosted. Nothing is internet-facing.

9. Where Things Are

Path	Contents
sql/01-04	Schema, RLS policies, masking views, demo data
sql/05, 07, 10, 14	The four walkthroughs. Each is readable top to bottom as an argument
sql/06, 08, 09	GoHighLevel bridge, review queue, review actions
sql/11-13	Deals, stage history, pipeline policies and seed
sql/99_local_logins.sql	Demo passwords. Local development only
web/	The demo: server.js and one HTML page
worker/src/	The GoHighLevel integration worker
worker/test/	63 tests
worker/tools/	The webhook capture tool
docs/	This document and the GoHighLevel interface specification, with their generators
README.md	The same ground in more technical detail, including the reasoning behind each design decision

There is no INSTALL file; section 2 of this document and the README's opening section cover installation. Both PDFs in docs/ are generated by committed scripts rather than written by hand, so they can be regenerated as the system changes and cannot quietly drift out of date.